Proyecto: Servidor nivel producción en Azure para un juego por turnos

22 oct 2026

Ing. Jonathan Solís Parajeles

Fecha de Entrega: 12 de Noviembre

Resumen

Los equipos diseñan, construyen y despliegan en Azure el servidor de un juego multijugador por turnos creado por ellos mismos. El servidor debe alojar muchas partidas simultáneas e independientes, cada una con al menos 2 jugadores, y mantener el estado de cada partida consistente aunque los jugadores se desconecten, reintenten acciones o una instancia del servidor se reinicie.

El juego es el pretexto, no el producto: su diseño, su arte y qué tan divertido es no se evalúan. Se evalúa el servidor: qué tan bien está construido, qué tan seguro es, qué tan eficiente es y qué tanto sigue los estándares de la industria. La entrega es el acceso al repositorio del servidor y su documentación.

Objetivos de aprendizaje

Al finalizar, el estudiante podrá:

1. Diseñar un servidor autoritativo, que valida cada jugada y es la única fuente de verdad del estado de la partida.
2. Gestionar el ciclo de vida de muchas sesiones concurrentes: creación, unión, inicio, turnos, desconexión, reconexión, fin y limpieza.
3. Diseñar un protocolo de comunicación en tiempo real y documentarlo como contrato, junto con una API REST para todo lo que no es tiempo real.
4. Resolver problemas de concurrencia: jugadas simultáneas, mensajes duplicados o fuera de orden, y varias instancias del servidor atendiendo la misma partida.
5. Aplicar seguridad de punta a punta: autenticación, autorización por partida, validación de mensajes, límites de tasa y protección contra trampas.
6. Medir y optimizar el servidor con pruebas de carga: latencia por jugada, conexiones simultáneas y consumo de recursos por partida.
7. Aprovisionar la infraestructura como código y desplegar en Azure mediante CI/CD, con observabilidad desde el primer despliegue.
8. Documentar y defender las decisiones de arquitectura.

El juego: lo diseña el equipo

Cada equipo inventa su propio juego. Las únicas restricciones son estas:

- **Por turnos:** en todo momento la partida sabe a quién le toca jugar. También se aceptan turnos simultáneos: todos envían su jugada y la ronda se resuelve al cerrarse.
- **Multijugador:** mínimo 2 jugadores humanos por partida; el máximo lo define el equipo.

- **Original:** puede inspirarse en mecánicas o temas de juegos existentes, pero no puede ser una recreación 1 a 1 de uno (por ejemplo, ajedrez, UNO o Battleship con otro nombre). Cambiar nombres o gráficos no basta; las reglas deben ser propias.
- **Reglas verificables:** el servidor debe poder decidir sin ambigüedad si una jugada es válida y cuándo termina la partida.
- **Información oculta (recomendada):** cartas en la mano, niebla de guerra u órdenes secretas. No es obligatoria, pero obliga a resolver un problema real de seguridad: no enviar a un jugador lo que no debe ver.

Requisitos funcionales del servidor

Estas son las capacidades mínimas. Los nombres de endpoints y mensajes los define el equipo en sus contratos.

Capacidad	Qué debe hacer	Reglas
Usuarios	Registro, inicio y cierre de sesión, renovación del token, perfil	Tokens de corta duración con refresh token rotativo, o un proveedor OIDC externo
Salas	Crear partida pública o privada (código de invitación), listar, unirse, salir, marcar «listo»	Inicia solo con 2 o más jugadores listos; la capacidad máxima se respeta aunque varios se unan a la vez
Emparejamiento	Cola rápida que agrupa jugadores en partidas nuevas	Un jugador nunca queda asignado a dos partidas; tiempo máximo de espera configurable
Turnos	Enviar jugada, recibir el nuevo estado y saber a quién le toca	Solo juega quien tiene el turno; jugada inválida rechazada con un error tipado; temporizador por turno con acción por defecto
Conexión	Detectar caídas (heartbeat), reconectar y recuperar el estado actual	Periodo de gracia configurable; al vencer, se aplica la regla de abandono del juego
Fin de partida	Declarar ganador o empate, registrar el resultado	Resultado inmutable; cada partida puede reproducirse jugada por jugada
Historial	Partidas jugadas y resultados por usuario	Paginado; un usuario solo ve el detalle de sus partidas
Administración	Ver partidas activas y conexiones, terminar una partida, suspender un usuario	Solo el rol administrador; cada acción queda auditada

Reglas del servidor.

- **Servidor autoritativo:** el cliente envía intenciones («jugar la carta 3»), nunca resultados («ahora tengo 10 puntos»). El servidor valida cada intención contra las reglas y el estado actual.
- **Orden por partida:** las jugadas de una misma partida se procesan una a la vez, y cada evento lleva un número de secuencia. Una jugada hecha sobre una versión vieja del estado se rechaza.
- **Idempotencia:** cada jugada lleva un identificador generado por el cliente. Si el cliente la reenvía tras perder la señal, no se aplica dos veces.
- **Vista por jugador:** cada jugador recibe solo lo que le corresponde ver. Ningún mensaje, endpoint ni log accesible filtra información oculta de otro jugador.
- **Durabilidad:** si una instancia se reinicia o se publica una versión nueva, las partidas en curso continúan y ninguna jugada confirmada se pierde.
- **Aislamiento:** una excepción en una partida no afecta a las demás ni tumba el proceso.
- **Limpieza:** las partidas abandonadas o inactivas expiran y liberan sus recursos automáticamente.
- **Reloj del servidor:** los temporizadores y el azar se resuelven en el servidor, nunca en el cliente. Si el azar decide el resultado, se usa un generador criptográficamente seguro.

Requisitos técnicos

Todos los requisitos siguientes son obligatorios; cómo se cumplen es decisión de diseño del equipo.

Lenguaje y stack.

Un lenguaje con tipado estático y su modo estricto activado: C# sobre .NET 8+ (nullable activado, warnings como errores), TypeScript sobre Node 20+ (strict: true, sin any), Go 1.22+, Java o Kotlin 21, o Rust. El motor de reglas del juego vive en un módulo de dominio puro, sin dependencias de red, base de datos ni framework.

Comunicación y contratos.

- **Tiempo real:** WebSocket seguro (wss://), directo o mediante SignalR, Socket.IO o Azure Web PubSub. No se permite consultar periódicamente (polling) para saber si cambió el turno.
- **Protocolo:** mensajes versionados y documentado con AsyncAPI 3.0: cada tipo de mensaje con su esquema, ejemplos y errores posibles.
- **API REST:** usuarios, salas, historial y administración, documentada con OpenAPI 3.1 y publicada en /docs. Los errores siguen RFC 9457 Problem Details.
- **Validación:** cada mensaje entrante se valida contra su esquema (tipos, rangos y tamaño máximo) antes de llegar al dominio. Un mensaje inválido se rechaza sin cerrar la partida.
- **Conexiones:** heartbeat (ping/pong) y cierre ordenado de conexiones, con códigos de cierre documentados.

Seguridad.

- **Autenticación:** JWT firmados, emitidos por un proveedor OIDC (Entra External ID, Auth0 u otro) o por el propio servidor, con contraseñas en Argon2id o bcrypt. La conexión WebSocket se autentica en el handshake, por ejemplo con un ticket de un solo uso y corta duración; si no se autentica, se cierra.

- **Autorización:** en cada endpoint y en cada mensaje, solo los participantes actúan en su partida, solo juega quien tiene el turno y solo el administrador usa las funciones de administración.
- **Límites:** tasa por usuario y por conexión (HTTP 429 con Retry-After), tamaño máximo de mensaje y número máximo de conexiones por usuario.
- **Transporte:** solo HTTPS y WSS, CORS y orígenes de WebSocket restringidos, y encabezados de seguridad HTTP.
- **Secretos:** en Key Vault, leídos con identidad administrada; ningún secreto en el repositorio ni en la configuración en texto plano.
- **Cadena de suministro:** escaneo de dependencias, de código y de secretos en CI (por ejemplo, Dependabot, CodeQL y gitleaks). La imagen del contenedor corre como usuario no root y no tiene vulnerabilidades críticas conocidas

Rendimiento y optimización.

- **Prueba de carga**
- **Metas:** latencia p95 menor a 200 ms entre que llega una jugada y que todos los jugadores de la partida reciben el nuevo estado, ninguna jugada perdida ni duplicada, y tasa de error menor a 1 %.
- **Capacidad:** reporte con CPU y memoria por partida activa y el punto de quiebre: cuántas partidas soporta una réplica antes de degradarse.
- **Mensajes compactos:** enviar solo los cambios o el estado mínimo necesario. El formato (JSON, MessagePack o Protobuf).
- **Optimización medida**

Arquitectura y despliegue en Azure

Esta es la topología de referencia. Un equipo puede sustituir un componente (por ejemplo, App Service con WebSockets en lugar de Container Apps, Cosmos DB en lugar de PostgreSQL, o Azure Web PubSub para las conexiones) si lo justifica.

Los jugadores se conectan por HTTPS y WSS a cualquier réplica del servidor en Container Apps. El estado caliente y la difusión de eventos entre réplicas pasan por Redis, las jugadas confirmadas se guardan en PostgreSQL y los secretos se leen de Key Vault con identidad administrada.

- **Infraestructura como código:** todos los recursos en Bicep o Terraform; un ambiente nuevo se levanta con una sola ejecución del pipeline.
- **CI/CD:** GitHub Actions o Azure DevOps ejecuta build, linter, pruebas, validación de contratos y escaneo de seguridad, y publica la imagen en el registro. Despliega a dev automáticamente y a prod con aprobación manual..
- **Publicaciones sin cortar partidas:** revisiones con división de tráfico y drenado de conexiones, y un rollback documentado de menos de 5 minutos.
- **Identidad:** identidad administrada para todo acceso entre servicios; ninguna cadena de conexión ni clave en la configuración.
- **Observabilidad:** OpenTelemetry hacia Application Insights; logs estructurados con el ID de la partida y del jugador, sin datos sensibles; métricas propias (partidas activas, conexiones, jugadas por segundo, latencia por jugada); sondas /health/live y /health/ready; al menos 2 alertas.

- **Costo:** dentro del crédito de Azure for Students, con una estimación de costo mensual en la documentación. Los recursos de dev se apagan o escalan a cero cuando no se usan.

Entregables

La entrega final tiene dos partes. El profesor evalúa la versión etiquetada v1.0.0 en la fecha de entrega.

- **Acceso al repositorio:** GitHub, con acceso de lectura para el profesor. Contiene el código del servidor, la infraestructura como código, los pipelines, los contratos AsyncAPI y OpenAPI, las pruebas, los scripts de carga, el bot y el cliente de prueba. El README explica cómo correr el servidor localmente en menos de 10 minutos (por ejemplo, con docker compose up), cómo correr las pruebas, y da la URL pública en Azure, que debe estar disponible durante el periodo de revisión.
- **Documentación**

Rúbrica de evaluación

Criterio	Peso	Qué obtiene la nota completa
Arquitectura y calidad de construcción	20%	Dominio aislado del transporte y la persistencia; código limpio y consistente
Sesiones y concurrencia	20%	Muchas partidas simultáneas sin interferir entre sí; turnos, temporizadores, reconexión y abandono correctos; jugadas ordenadas e idempotentes; funciona con varias réplicas y sobrevive a reinicios
Seguridad	20%	Autenticación y autorización por partida en HTTP y en tiempo real; cada mensaje validado; límites de tasa; sin filtración de información oculta; secretos en Key Vault; modelo de amenazas
Rendimiento y optimización	15%	Prueba de carga reproducible que cumple las metas; punto de quiebre medido; al menos una optimización con números antes y después
Despliegue en Azure, IaC y CI/CD	10%	Ambiente reconstruido desde cero con el pipeline; identidad administrada; despliegues sin cortar partidas; rollback probado
Estándares, contratos y pruebas	10%	AsyncAPI y OpenAPI completos y sincronizados con el código; Problem Details; pruebas unitarias, de integración, concurrencia, seguridad y resiliencia en verde en CI
Documentación y observabilidad	5%	Documentación clara y completa; logs y trazas correlacionados por partida; métricas propias, sondas de salud y alertas

El juego en sí (reglas, gráficos, diversión y cliente) no tiene nota; solo se verifica que cumpla las restricciones de la sección «El juego». No se acepta un juego que sea una recreación 1 a 1 de otro. Cualquier secreto incluido en el repositorio limita el criterio de seguridad a un 50 %. Las entregas tardías pierden un 10 % de la nota por día.